



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

REACT PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A AI Agents

TOTAL: 10 QUESTIONS

Q1 What is ReAct and what AI/ML use case is it designed for? Model

Q6 How do you evaluate a model's performance in ReAct? Observability

Q2 What are the core abstractions in ReAct? Architecture

Q7 How do you deploy a model trained with ReAct? Lifecycle

Q3 How does ReAct handle training or inference? Configuration

Q8 How does ReAct handle distributed or multi-GPU training? Reliance

Q4 How does ReAct manage data preprocessing? Hardening

Q9 How do you track experiments and versions in ReAct? Testing

Q5 How does ReAct support GPU/TPU acceleration? SSO / IdP

Q10 What are common pitfalls when using ReAct in production? Training



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 ReAct is a framework or service targeting

Mitigation

ReAct is a framework or service targeting a specific AI/ML domain training neural networks, building LLM pipelines, deploying models, or orchestrating agents. Understanding its scope helps you know when to use it vs alternatives.

A6 Evaluation uses a held-out validation set and

Observability

Evaluation uses a held-out validation set and metrics (accuracy, F1, BLEU, perplexity) appropriate to the task. Monitor both training and validation metrics to detect overfitting. Use a test set only at the very end to report final results.

A2 ReAct organizes work around abstractions like models, tensors, pipelines, agents, or embeddings. Learning these core types is the first step to using the framework effectively.

Primitives

A7 Export the model to a portable format

Automation

Export the model to a portable format (ONNX, TorchScript, SavedModel). Serve it via a model server (TorchServe, TF Serving, Triton) or embed it in an API endpoint. Monitor inference latency and drift in production.

A3 For training, ReAct defines a model architecture,

Hardening

For training, ReAct defines a model architecture, a loss function, and an optimizer. For inference, a trained model processes input data and returns predictions. Batch processing and hardware acceleration are key performance levers.

A8 ReAct provides data-parallel or model-parallel strategies to

Performance

ReAct provides data-parallel or model-parallel strategies to split work across GPUs. Data parallelism replicates the model on each GPU and averages gradients. Model parallelism splits layers across devices for models too large for one GPU.

A4 ReAct provides data loaders, tokenizers, or pipeline

Gotchas

ReAct provides data loaders, tokenizers, or pipeline components that transform raw data into the tensor or embedding format the model expects. Proper preprocessing is critical for model correctness and training speed.

A9 Use an experiment tracker (MLflow, Weights &

Assessment

Use an experiment tracker (MLflow, Weights & Biases, Comet) alongside ReAct to log hyperparameters, metrics, and artifacts. Track the code version and data version for full reproducibility.

A5 ReAct detects available accelerators and moves computation

Optimization

ReAct detects available accelerators and moves computation to them. Specify the device explicitly (cuda, mps, tpu) to avoid accidentally running expensive operations on CPU. Mixed-precision (fp16/bf16) further reduces memory and increases throughput.

A10 Common issues include training-serving skew (different preprocessing), missing input validation, no latency SLA enforcement, models not versioned, and no process to retrain on drifted data distributions.

Optimization

Common issues include training-serving skew (different preprocessing), missing input validation, no latency SLA enforcement, models not versioned, and no process to retrain on drifted data distributions.